Metro Ticket Booking System

Project Logic & Business Rules Specification (Planning Phase)

PLATFORM TARGET	APPLICATION SCOPE	DOCUMENT VERSION
ServiceNow App Engine	Scoped App (x_snc_metro_book)	v1.0 Final

1. High-Level Project Logic Overview

The business logic spans four sequential phases across the ticket lifecycle: Fare Calculation, Payment Verification & Token Issuance, Gate Scan Validation (<200ms), and Background Reconciliation.

2. Core Operational Logic Modules

Module 1: Dynamic Fare & Tariff Calculation Logic

Inputs: `Origin Station`, `Destination Station`, `Passenger Category`, `Travel Timestamp`.

- **Hop Distance:** Calculates total station index differential: *N = |Dest Index - Origin Index|*.
- **Base Fare Lookup:** Queries `sn_metro_fare_matrix` for station hop tier.
- **Peak-Hour Multiplier:** Evaluates time windows (08:00–10:00 AM & 05:00–07:00 PM). If active, applies a 1.25x multiplier.
- **Passenger Discount:** Applies user category discount (Student: 20%, Senior: 30%).

$$\textit{Final Fare} = (\textit{Base Fare} \times \textit{Peak Multiplier}) \times (1 - \textit{Discount Rate})$$

Module 2: Ticket Generation & Anti-Fraud Logic

Trigger: Payment Webhook callback returns `SUCCESS`.

- **Security Hash:** Generates HMAC-SHA256 signature combining `sys_id` + `commuter_id` + `timestamp` + `valid_until` + `secret_key`.
- **TOTP Dynamic QR:** Appends a 30-second rotating Time-Based One-Time Password seed to prevent screenshot sharing.
- **Validity Window:** Sets ticket expiry duration: *Valid Until = Issue Time + 3 Hours*.

Module 3: Real-Time Gate Scan Logic (<200ms Execution)

Inputs: `station_id`, `qr_token`, `gate_type` (ENTRY / EXIT).

- **Entry Scan Processing:**
  - Queries indexed ticket key; checks state is `Active` and time is within validity window.
  - If valid: Returns `{"gate_action": "OPEN", "status": "ALLOW"}`.
  - *Async Action:* Offloads background update to transition ticket state to `In-Transit`.
- **Exit Scan Processing:**
  - Verifies ticket state is `In-Transit` and station matches destination.
  - If commuter over-traveled, triggers penalty calculation prompt before exit clearance.

- If valid: Returns `{"gate_action": "OPEN", "status": "ALLOW"}` and updates state to `Closed`.

Module 4: Exception Handling & Self-Healing Refund Logic

- **Payment Timeout Rule:** Initiates a 60-second Flow Designer timer upon pending payment creation.
- **Auto-Refund Execution:** If callback is missed or ticket generation fails after payment, an automated Integration Hub REST step executes a refund request via payment gateway API.

Module 5: Background Expiration & Cleanup Engine

A Scheduled Script Execution executes every 15 minutes to mark unused overdue tickets as `Expired`.

```
var gr = new GlideRecord('x_snc_metro_book_ticket');
gr.addQuery('state', 'Active');
gr.addQuery('valid_until', '<', gs.nowDateTime());
gr.query();
while(gr.next()) {
    gr.setValue('state', 'Expired');
    gr.update();
}
```

3. Core Logic Rules Mapping Table

Logic Event	ServiceNow Component	Execution Type	Latency Target
Fare Pricing Lookups	Script Include + Decision Tables	Synchronous	< 50 ms
Gate Scanner Validation	Scripted REST API (`POST`)	Synchronous	< 150 ms
Ticket State Updates	Async Business Rules	Asynchronous	Non-blocking
Payment Confirmation	Integration Hub Webhook	Async Event	< 2 s
Overdue Ticket Cleanup	Scheduled Script Execution	Background Job	Low Priority